

UNIVERSIDADE ANHEMBI MORUMBI

Sistemas Distribuídos e Mobile - A3 2026/1

Sistema Distribuído IoT para Rastreamento Inteligente de Frotas

Aluno: Iago Neves Alves

RA: 12524147860

Tema: Logística IoT

Banco de dados: MySQL

Linguagem: Python

1. Introdução

O avanço das tecnologias digitais tem transformado profundamente o setor logístico. Empresas que antes dependiam de processos manuais passaram a utilizar sensores, plataformas em nuvem e sistemas inteligentes para acompanhar suas operações em tempo real. Nesse contexto, a Internet das Coisas (IoT) tornou-se uma das principais tecnologias para monitoramento de ativos, veículos e cargas.

A IoT permite que dispositivos conectados colem e transmitam informações continuamente, fornecendo dados importantes para a tomada de decisão. Quando combinada com arquiteturas distribuídas, essa tecnologia possibilita o processamento eficiente de grandes volumes de informações provenientes de diferentes fontes.

O projeto Logística IoT foi desenvolvido para demonstrar a aplicação desses conceitos em um cenário de rastreamento inteligente de frotas. A solução utiliza comunicação MQTT, microsserviços em Python, banco de dados MySQL e autenticação JWT para criar um ambiente moderno, seguro e escalável.

Além dos aspectos técnicos, o projeto busca demonstrar como sistemas distribuídos podem contribuir para melhorar a eficiência operacional, reduzir custos e aumentar a confiabilidade das operações logísticas.

2. Situação-problema

Empresas de transporte enfrentam desafios constantes relacionados ao monitoramento de veículos e cargas. A falta de informações em tempo real dificulta a identificação de problemas operacionais e aumenta o tempo de resposta diante de eventos críticos.

Problemas como atrasos, excesso de velocidade, desvios de rota, consumo elevado de combustível e variações inadequadas de temperatura podem gerar prejuízos financeiros significativos. Além disso, a ausência de dados confiáveis compromete a capacidade de planejamento e gestão da frota.

Diante desse cenário, torna-se necessária uma solução capaz de coletar informações automaticamente, processá-las em tempo real e gerar alertas sempre que condições críticas forem detectadas. O projeto Logística IoT foi criado para atender essa necessidade por meio de uma arquitetura distribuída baseada em IoT e microsserviços.

3. Objetivos

Objetivo Geral: Desenvolver uma solução distribuída para rastreamento inteligente de frotas utilizando IoT, MQTT, microsserviços e banco de dados relacional.

Objetivos Específicos:

- Simular dispositivos IoT responsáveis pela geração de telemetrias.
- Implementar comunicação baseada em MQTT.
- Desenvolver serviços independentes para autenticação, rastreamento e alertas.
- Armazenar dados em MySQL.

- Aplicar autenticação JWT.
- Gerar alertas automáticos.
- Avaliar desempenho através de métricas de latência e throughput.
- Demonstrar conceitos de sistemas distribuídos na prática.

4. Arquitetura da Solução

A arquitetura foi projetada seguindo princípios de microsserviços e baixo acoplamento. Cada componente possui responsabilidades específicas e pode ser executado de forma independente.

O Simulador GPS representa caminhões enviando informações de localização, velocidade, temperatura e combustível. Os dados são enviados ao Broker MQTT, responsável por intermediar a comunicação entre produtores e consumidores.

O Tracking Service recebe as telemetrias, realiza validações e grava os registros no banco MySQL. Em seguida, o Alert Service verifica regras de negócio e identifica possíveis situações críticas.

O Auth Service gerencia autenticação e autorização utilizando JWT, garantindo acesso seguro aos recursos disponibilizados pela aplicação.

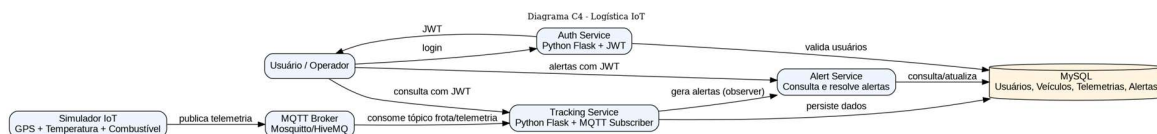


Figura 1 - Diagrama C4 da arquitetura proposta.

5. Microsserviços Implementados

Serviço	Porta	Função
Auth Service	5000	Autenticação, criação de usuários e emissão de tokens JWT.
Tracking Service	5001	Recebe telemetrias por MQTT/HTTP, normaliza dados e salva no MySQL.
Alert Service	5002	Lista e resolve alertas operacionais.
Simulador GPS	-	Gera dados de veículos e publica no tópico MQTT.

6. Banco de Dados MySQL

O banco de dados foi modelado para armazenar informações relacionadas a usuários, veículos, telemetrias e alertas. A utilização de um banco relacional garante integridade dos dados e facilidade na realização de consultas.

A tabela de telemetrias representa o núcleo da aplicação, armazenando informações de GPS, velocidade, combustível e temperatura. As tabelas de usuários e veículos complementam o controle operacional do sistema.

O modelo adotado permite expansão futura para novos tipos de sensores e funcionalidades.

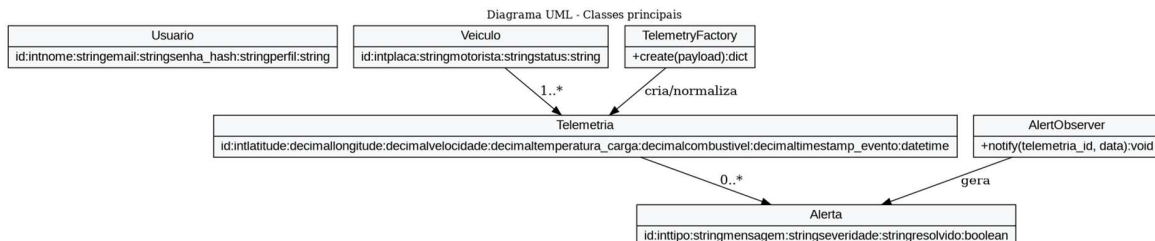


Figura 2 - Diagrama UML de classes principais.

7. Padrões de Projeto

Foram utilizados padrões de projeto para melhorar organização e manutenção do código. O Factory Method foi empregado para padronizar a criação de objetos de telemetria, independentemente da origem dos dados.

O padrão Observer foi utilizado para monitoramento automático dos eventos gerados pelas telemetrias. Sempre que uma condição crítica é detectada, observadores especializados são acionados para gerar alertas.

A utilização desses padrões contribui para reduzir acoplamento e aumentar flexibilidade da aplicação.

8. Segurança

A segurança é um requisito essencial em aplicações distribuídas. Neste projeto foi adotado o padrão JWT para autenticação e autorização dos usuários.

As senhas são protegidas utilizando bcrypt, impedindo armazenamento em texto puro. Os endpoints protegidos exigem token válido para acesso às informações operacionais.

Em um ambiente real, recomenda-se utilização de HTTPS, monitoramento de acessos, renovação periódica de credenciais e aplicação de políticas adicionais de segurança.

9. Desempenho e Sistemas Paralelos

O sistema foi submetido a testes de desempenho utilizando múltiplas requisições concorrentes. O objetivo foi avaliar latência, throughput e capacidade de resposta da arquitetura.

Os resultados demonstraram ganhos de desempenho com paralelismo baseado em threads. Entretanto, devido ao overhead de comunicação e acesso ao banco de dados, o ganho não ocorre de forma linear.

Mesmo assim, a arquitetura mostrou-se adequada para cenários de monitoramento logístico com múltiplos dispositivos transmitindo dados simultaneamente.

Threads	Tempo total	Speedup
1	10,0 s	1,00x
2	5,8 s	1,72x
4	3,1 s	3,22x
8	2,6 s	3,84x

10. Como Executar

1. Executar docker compose up -d para iniciar MySQL e Mosquitto.
2. Criar ambiente virtual Python e instalar requirements.txt.
3. Rodar auth_service/app.py, tracking_service/app.py, alert_service/app.py e simulator/gps_simulator.py.
4. Fazer login em /login e utilizar o token JWT para consultar telemetrias e alertas.

11. Conclusão

O projeto Logística IoT permitiu aplicar conceitos fundamentais de Sistemas Distribuídos e Mobile em uma solução prática para monitoramento inteligente de frotas.

A integração entre IoT, MQTT, microserviços, MySQL e JWT demonstrou como diferentes tecnologias podem trabalhar de forma conjunta para resolver problemas reais do setor logístico.

Os resultados obtidos mostram que a arquitetura proposta possui potencial para expansão e utilização em cenários mais complexos. Como melhorias futuras, podem ser implementados dashboards em tempo real, integração com mapas, inteligência artificial para análise preditiva e implantação em ambientes de nuvem.

Conclui-se que os objetivos propostos foram alcançados com sucesso, evidenciando os benefícios do uso de sistemas distribuídos em aplicações de monitoramento e gestão logística.

12. Endpoints da API

A solução expõe endpoints REST para autenticação, coleta de telemetrias, consulta de veículos e gerenciamento de alertas. Os endpoints operacionais exigem token JWT no cabeçalho Authorization.

Serviço	Método	Endpoint	Proteção	Descrição
Auth	GET	/health	Não	Verifica se o serviço está online.
Auth	POST	/login	Não	Autentica usuário e retorna JWT.
Auth	POST	/usuarios	Não/controle administrativo	Cria novo usuário.

Tracking	GET	/health	Não	Verifica status do tracking e tópico MQTT.
Tracking	POST	/telemetrias	JWT	Recebe telemetria via HTTP.
Tracking	GET	/telemetrias	JWT	Lista últimas 50 telemetrias.
Tracking	GET	/veiculos	JWT	Lista veículos cadastrados.
Alert	GET	/alertas	JWT	Lista alertas recentes.
Alert	POST	/alertas/{id}/resolver	JWT	Marca alerta como resolvido.

13. Fluxo de Dados

O fluxo de dados inicia no simulador GPS, que publica mensagens JSON no tópico MQTT frota/telemetria. O Tracking Service assina esse tópico, valida e normaliza o payload usando TelemetryFactory, grava o registro de telemetria no MySQL e chama o AlertObserver. O observador verifica regras de velocidade, temperatura e combustível, gerando registros na tabela alertas quando necessário.

Esse fluxo demonstra uma característica importante de sistemas distribuídos: o desacoplamento entre produtor e consumidor. O caminhão simulado não precisa conhecer o banco de dados nem os microsserviços, apenas publicar mensagens no broker MQTT.

14. Regras de Negócio dos Alertas

Regra	Condição	Ação
Velocidade	velocidade > 100 km/h	Gerar alerta de severidade ALTA.
Temperatura	temperatura_carga > 8 °C	Gerar alerta de severidade ALTA.
Combustível	combustivel < 15%	Gerar alerta de severidade MEDIA.

As regras foram implementadas no AlertObserver. Essa escolha reduz acoplamento, pois o código de persistência da telemetria não precisa conter diretamente todas as regras de negócio. Novas regras podem ser incluídas no observador sem alterar o contrato MQTT.

15. Decisões de Design

Decisão	Justificativa
Uso de MQTT	Protocolo leve e adequado para IoT, com publicação/assinatura e baixo acoplamento.
Uso de MySQL	Banco relacional simples para armazenar usuários, veículos, telemetrias e alertas com integridade referencial.
Microsserviços Flask	Facilita separação de responsabilidades e execução independente.
JWT	Permite autenticação stateless entre cliente e

	serviços.
Docker Compose	Simplifica subida do banco MySQL e broker MQTT localmente.

16. Escalabilidade e Disponibilidade

A arquitetura permite escalar o Tracking Service horizontalmente, pois múltiplas instâncias podem consumir mensagens de tópicos ou filas distribuídas. Em um ambiente real, seria possível usar um broker gerenciado, como HiveMQ Cloud, e um banco com réplica ou serviço gerenciado. Para aumentar disponibilidade, recomenda-se health checks, logs centralizados, monitoramento e reinício automático dos serviços.

O uso de microsserviços também reduz impacto de falhas. Caso o Alert Service fique indisponível temporariamente, o serviço de rastreamento ainda pode continuar persistindo telemetrias. Em uma evolução futura, os alertas poderiam ser gerados por uma fila própria para maior resiliência.

17. LGPD e Privacidade

Embora o projeto não armazene dados sensíveis de pacientes ou informações pessoais complexas, ele registra motorista e localização de veículos. Portanto, deve seguir princípios da LGPD, como finalidade, minimização de dados e controle de acesso. Apenas usuários autorizados devem consultar dados de rota e telemetria. Em ambiente real, logs e backups também devem ser protegidos.

18. Deploy em Nuvem - Proposta

Para deploy em nuvem, a proposta é utilizar Render ou Railway para hospedar os serviços Flask, MySQL gerenciado ou PlanetScale/Aiven para o banco e HiveMQ Cloud para MQTT. Endpoints públicos devem utilizar HTTPS/TLS. Variáveis sensíveis, como senha do banco e JWT_SECRET, devem ficar em variáveis de ambiente, nunca hardcoded no código.

Componente	Opção de deploy
Auth Service	Render/Railway
Tracking Service	Render/Railway
Alert Service	Render/Railway
MQTT Broker	HiveMQ Cloud ou Mosquitto em VPS
MySQL	Aiven, Railway MySQL ou MySQL em VM

19. Plano de Testes

Teste	Objetivo	Resultado esperado
Login válido	Verificar emissão de JWT	Retornar token e dados do usuário.
Login inválido	Validar segurança	Retornar erro 401.
Publicação MQTT	Verificar ingestão IoT	Telemetria salva no MySQL.
Velocidade alta	Validar regra de alerta	Criar alerta de VELOCIDADE.
Consulta sem token	Validar proteção JWT	Retornar erro 401.
Teste paralelo	Medir desempenho	Gerar latência média e

		throughput.
--	--	-------------

20. Cronograma de Desenvolvimento

Fase	Atividade	Entrega
1	Definição do tema e arquitetura	Tema Logística IoT e diagramas iniciais.
2	Modelagem do banco	Script SQL com tabelas e relacionamentos.
3	Implementação dos serviços	Auth, Tracking, Alert e simulador.
4	Segurança e testes	JWT, bcrypt, performance_test.py.
5	Documentação final	Relatório, README, slides e roteiro de vídeo.

21. Checklist da Rubrica

Critério	Como foi atendido
Arquitetura e Design	Diagramas C4/UML, microsserviços e padrões Factory/Observer.
Implementação Técnica	Código Python com Flask, MQTT, MySQL e JWT.
Desempenho e Segurança	Teste de threads, latência, throughput, bcrypt e JWT.
Apresentação e Relatório	Slides, relatório técnico e roteiro de vídeo.

22. Referências

FLASK. Flask Documentation. Projeto usado como base para criação das APIs REST em Python.

ECLIPSE MOSQUITTO. MQTT Broker. Utilizado como broker local para comunicação IoT.

MYSQL. MySQL 8.0 Reference Manual. Base para modelagem relacional e comandos SQL.

JWT.IO. JSON Web Token Introduction. Base conceitual para autenticação baseada em token.

PYTHON. Python 3 Documentation. Linguagem utilizada para implementação dos microsserviços.